

TP Java – Programmation Orientée Objet

Matisse

kra

bts sio

Sommaire

Objectif	2
EXERCICE 1 : LA CLASSE LIVRE	2
1. Définition de la classe Livre	2
2. Constructeur avec saisie utilisateur	2
3. Méthodes d'accès (getters et setters)	2
4. Méthode afficher()	3
5. Programme de test	3
7. Difficultés rencontrées	4
8. code complets	4
9. résultat de la console	5
Exercice 2 : Classe Compte	5
1.1. La classe Client	5
1.2. Méthodes d'accès (getters / setters)	6
1.3. Constructeur complet	6
1.4. Constructeur partiel (avec saisie utilisateur)	7
1.5. Méthode Afficher()	8
2. La classe Compte	8
2.1 Définition des attributs	8
2.2 methode d'accès	9
2.3 Constructeur	9
3. Méthodes du compte	9
3.1 Crédit simple	9
3.2 Créditer depuis un autre compte	10
3.3 Débit simple	10
3.4 Débit vers un autre compte	10
4. Programme de test	10
résultat	11
Conclusion	11

Objectif

Le but du TP était d'apprendre à structurer une classe avec ses attributs, son constructeur, ses méthodes d'accès et une méthode d'affichage. Enfin, j'ai réalisé un programme de test permettant d'instaurer un objet et de vérifier le bon fonctionnement de la classe.

Ce TP m'a permis de renforcer ma compréhension de la programmation orientée objet, notamment l'encapsulation et la manipulation d'objets en Java.

EXERCICE 1 : LA CLASSE LIVRE

1. Définition de la classe Livre

La première étape consistait à définir une classe nommée Livre contenant trois attributs :

- titre : chaîne de caractères
- auteur : chaîne de caractères (nom complet de l'auteur)
- prix : nombre réel (j'ai utilisé *double* pour plus de précision)

Ces attributs ont été déclarés en *private* pour respecter l'encapsulation, c'est-à-dire la protection des données internes de la classe.

```
// ---- ATTRIBUTS ----  
private String titre;  
private String auteur;  
private double prix;
```

2. Constructeur avec saisie utilisateur

J'ai ensuite créé un constructeur sans paramètre. Celui-ci initialise les attributs en demandant directement les valeurs à l'utilisateur. Pour cela, j'ai utilisé un objet Scanner, qui permet de récupérer :

- le titre du livre (*nextLine*)
- l'auteur (*nextLine*)
- le prix (*nextDouble*)

Ce constructeur rend la création d'un objet Livre interactive.

```
// ---- CONSTRUCTEUR AVEC SAISIE UTILISATEUR ----  
public livre() {  
    try (Scanner scanner = new Scanner(System.in)) {  
  
        System.out.print("Entrez le titre du livre : ");  
        this.titre = scanner.nextLine();  
  
        System.out.print("Entrez le nom complet de l'auteur : ");  
        this.auteur = scanner.nextLine();  
  
        System.out.print("Entrez le prix du livre : ");  
        this.prix = scanner.nextDouble();  
  
    }  
}
```

3. Méthodes d'accès (getters et setters)

Afin de respecter la notion d'encapsulation, j'ai ajouté des méthodes d'accès, aussi appelées *getters* et *setters* :

- Les getters permettent de récupérer les valeurs
- Les setters permettent de les modifier

Ces méthodes assurent un accès contrôlé aux attributs privés de la classe.

```
// ---- methode d'accès ----  
public String getTitre() {  
    return titre;  
}
```

```
public String getAuteur() {  
    return auteur;  
}  
  
public double getPrix() {  
    return prix;  
}  
  
public void setTitre(String titre) {  
    this.titre = titre;  
}  
  
public void setAuteur(String auteur) {  
    this.auteur = auteur;  
}  
  
public void setPrix(double prix) {  
    this.prix = prix;  
}
```

4. Méthode afficher()

J'ai également ajouté une méthode **afficher()**.

Son rôle est d'afficher proprement les informations du livre :

- Titre
- Auteur
- Prix

L'affichage se fait à l'aide de plusieurs `System.out.println()`.

```
// ---- METHODE D'AFFICHAGE ----  
public void afficher() {  
    System.out.println("----- Informations du livre -----");  
    System.out.println("Titre : " + titre);  
    System.out.println("Auteur : " + auteur);  
    System.out.println("Prix : " + prix + "€");  
}
```

5. Programme de test

Pour valider le fonctionnement de ma classe, j'ai écrit une classe de test nommée `TestLivre`.

Ce programme :

1. affiche un message d'introduction,
2. crée un objet livre (ce qui déclenche le constructeur),
3. appelle la méthode `afficher()` pour montrer les informations saisies.

Cela m'a permis de vérifier que la classe fonctionnait correctement.

```
class TestLivre {  
    public static void main(String[] args) {  
  
        System.out.println("=== Test de la classe Livre ===");  
  
        // Création d'un objet Livre avec saisie utilisateur  
        livre livre1 = new livre();  
    }  
}
```

```
// Affichage des informations du livre
System.out.println("\nVoici les informations du livre saisi :");
livre1.afficher();
}
}
```

7. Difficultés rencontrées

- Gestion du Scanner

Il fallait faire attention à l'ordre des saisies entre `nextLine()` et `nextDouble()` pour éviter les erreurs de lecture.

- Organisation du projet Java

Au début, VS Code signalait que les fichiers étaient "hors projet". J'ai dû les placer dans un dossier `src` pour que Java les reconnaisse correctement.

- Encapsulation

Comprendre pourquoi les attributs sont en `private` et pourquoi on doit passer par des `getters/setters`.

8. code complets

```
src > Classes > J livre.java > TestLivres
1 package Classes;
2 import java.util.Scanner;
3
4
5 public class livre {
6
7     // ---- ATTRIBUTS ----
8     private String titre;
9     private String auteur;
10    private double prix;
11
12    // ---- CONSTRUCTEUR AVEC SAISIE UTILISATEUR ----
13    public livre() {
14        try (Scanner scanner = new Scanner(System.in)) {
15
16            System.out.print(s: "Entrez le titre du livre : ");
17            this.titre = scanner.nextLine();
18
19            System.out.print(s: "Entrez le nom complet de l'auteur : ");
20            this.auteur = scanner.nextLine();
21
22            System.out.print(s: "Entrez le prix du livre : ");
23            this.prix = scanner.nextDouble();
24        }
25    }
26
27    // ---- methode d'accès ----
28    public String getTitre() {
29        return titre;
30    }
31
32    public String getAuteur() {
33        return auteur;
34    }
35
36    public double getPrix() {
37        return prix;
38    }
39
40    public void setTitre(String titre) {
41        this.titre = titre;
42    }
43
44    public void setAuteur(String auteur) {
45        this.auteur = auteur;
```

```
46    }
47
48    public void setPrix(double prix) {
49        this.prix = prix;
50    }
51
52    // ---- METHODE D'AFFICHAGE ----
53    public void afficher() {
54        System.out.println(x: "---- Informations du livre ----");
55        System.out.println("Titre : " + titre);
56        System.out.println("Auteur : " + auteur);
57        System.out.println("Prix : " + prix + "€");
58    }
59 }
60
61 class TestLivres {
62     Run | Debug
63     public static void main(String[] args) {
64
65         System.out.println(x: "=== Test de la classe Livre ===");
66
67         // Création d'un objet Livre avec saisie utilisateur
68         livre livre1 = new livre();
69
70         // Affichage des informations du livre
71         System.out.println(x: "\nVoici les informations du livre saisi :");
72         livre1.afficher();
73     }
74 }
```

9. résultat de la console

```
=== Test de la classe Livre ===  
Entrez le titre du livre : moi  
Entrez le nom complet de l'auteur : moi  
Entrez le prix du livre : 19,99  
  
Voici les informations du livre saisi :  
----- Informations du livre -----  
Titre : moi  
Auteur : moi  
Prix : 19.99€
```

Exercice 2 : Classe Compte

1.1. La classe Client

J'ai commencé par créer une classe client contenant les informations nécessaires pour représenter un client :

- nom
- prenom
- tel
- cin

Ces attributs sont privés pour respecter l'encapsulation en Java.

```
/* --- Attributs --- */  
private String nom;  
private String prenom;  
private double tel;  
private String cin;
```

1.2.Méthodes d'accès (getters / setters)

J'ai créé les getters et setters pour permettre d'accéder et modifier proprement les données du client.

```
/* methode acces */
/* --- GETTERS --- */
public String getnom() {
    return nom;
}

public String getprenom() {
    return prenom;
}

public double gettel() {
    return tel;
}

public String getcin() {
    return cin;
}

/* --- SETTERS --- */
public void setnom(String nom) {
    this.nom = nom;
}

public void setprenom(String prenom) {
    this.prenom = prenom;
}

public void settel(double tel) {
    this.tel = tel;
}

public void setcin(String cin) {
    this.cin = cin;
}
```

1.3.Constructeur complet

J'ai ensuite créé un constructeur permettant d'initialiser tous les attributs d'un seul coup :
Ce constructeur est utile quand on connaît déjà toutes les informations du client.

```
/* --- CONSTRUCTEUR QUI INITIALISE TOUS LES ATTRIBUTS --- */

public client(String nom, String prenom, double tel, String cin) {

    this.nom = nom;

    this.prenom = prenom;
```

matisse kra
bts sio

```
        this.tel = tel;

        this.cin = cin;

    }
```

1.4.Constructeur partiel (avec saisie utilisateur)

Pour rendre le programme interactif, j'ai aussi créé un constructeur utilisant un Scanner, ce qui permet de demander les informations au clavier.Ce constructeur m'a permis d'apprendre à gérer la saisie clavier dans une classe.

```
/* --- CONSTRUCTEUR AVEC SAISIE UTILISATEUR --- */

public client(Scanner scanner) {

    System.out.print("Entrez votre Nom : ");

    this.nom = scanner.nextLine();

    System.out.print("Entrez votre Prénom : ");

    this.prenom = scanner.nextLine();

    System.out.print("Entrez votre CIN : ");

    this.cin = scanner.nextLine();

    System.out.print("Entrez votre numéro de téléphone : ");

    this.tel = scanner.nextDouble();

}
```

1.5.Méthode Afficher()

J'ai codé une méthode permettant d'afficher toutes les informations du client à l'écran. Cette méthode est utile pour vérifier que les données du client ont bien été enregistrées.

```
/* methode affichage */

public void afficheclient() {

    System.out.println("----- compte -----");

    System.out.println("Nom : " + nom);

    System.out.println("Prénom : " + prenom);

    System.out.println("cin : " + cin);

    System.out.println("tél : " + tel);

}
```

2. La classe Compte

2.1 Définition des attributs

La classe compte contient 4 informations importantes :

- un code de compte auto-incrémenté (static)
- un solde (lecture seule)
- un propriétaire (un objet de type client)
- un montant

Le compteur est géré grâce à une variable statique :

```
private static int compteur = 0; // compteur partagé pour générer les codes
private int code;                // numéro de compte (lecture seule)
private double solde;            // lecture seule
private client proprietaire;
private double montant;
```


2.2 methode d'accès

J'ai créé les getters pour permettre d'accéder et modifier proprement les données.

```
/* methode acces */
/* --- GETTERS --- */
public int getCode() {
    return code;
}

public double getSolde() {
    return solde;
}

public client getProprietaire() {
    return proprietaire;
}
```

2.3 Constructeur

Le constructeur d'un compte prend en paramètre un client et :

- incrémente automatiquement le numéro de compte
- fixe le solde à 0
- attache le client comme propriétaire

```
/* CONSTRUCTEUR compte */
public compte(client proprietaire) {
    compteur++;
    this.code = compteur; // auto-incrémenté
    this.solde = 0;
    this.proprietaire = proprietaire;
}
```

3. Méthodes du compte

Ce TP m'a demandé d'implémenter plusieurs fonctionnalités bancaires.

3.1 Crédit simple

Permet d'ajouter une somme au solde :

```
public void Crediter() {
    System.out.println("$ $ $montant a deposer$ $ $");
    this.montant= scanner.nextDouble();
    System.out.println("operation effectuer");
    solde += montant;
}
```

3.2 Créditer depuis un autre compte

```
// 2. Créditer depuis un autre compte  
public void Crediter(double montant, compte compteSource) {  
    compteSource.solde -= montant;  
    this.solde += montant;  
}
```

3.3 Débit simple

Même principe que Crediter(), mais en retirant de l'argent.

```
// 3. Débit simple  
public void Debiter() {  
    System.out.println("$ $ $montant a retirer $ $ $");  
    this.montant= scanner.nextDouble();  
    System.out.println("operation effectuer");  
    solde -= montant;  
}
```

3.4 Débit vers un autre compte

Comme pour Crediter(), mais dans l'autre sens :

```
// 4. Débit vers un autre compte  
public void Debiter(double montant, compte compteDest) {  
    this.solde -= montant;  
    compteDest.solde += montant;  
}
```

4. Programme de test

Pour terminer le TP, j'ai écrit une classe testcompte avec un main() permettant :

1. de créer un client
2. de créer un compte associé
3. d'afficher les infos
4. de tester les opérations (crédit/débit)
5. de réafficher l'état du compte après chaque opération

Ce test m'a permis de vérifier que toutes les classes fonctionnent ensemble.

```
package Compte;  
  
import java.util.Scanner;  
  
public class testcompte {  
    Run | Debug  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        client c1 = new client(scanner);  
  
        compte compte1 = new compte(c1);  
        compte1.AfficherResume();  
  
        compte1.Crediter();  
        compte1.AfficherResume();  
  
        compte1.Debiter();  
        compte1.AfficherResume();  
    }  
}
```

résultat

```
Entrez votre Nom : kra
Entrez votre Prénom : matisse
Entrez votre CIN : EEE112
Entrez votre numéro de téléphone : 0757576404
---- Résumé du Compte ----
Numéro : 1
Solde : 0.0 €
Propriétaire : kra matisse
Propriétaire : 7.57576404E8
Propriétaire : EEE112
$ $ $ montant a deposter$ $ $
1000
operation effectuer
---- Résumé du Compte ----
Numéro : 1
Solde : 1000.0 €
Propriétaire : kra matisse
Propriétaire : 7.57576404E8
Propriétaire : EEE112
$ $ $ montant a retirer$ $ $
500
operation effectuer
---- Résumé du Compte ----
Numéro : 1
Solde : 500.0 €
Propriétaire : kra matisse
Propriétaire : 7.57576404E8
Propriétaire : EEE112
```

Conclusion

Ce TP m'a appris plusieurs notions importantes de la POO en Java :

- créer et structurer une classe
- utiliser les getters et setters
- écrire plusieurs constructeurs
- gérer l'interaction entre objets

matisse kra
bts sio

- manipuler un compteur statique
- créer des méthodes métiers (crédit, débit, transfert)
- tester un programme avec une classe main

J'ai aussi rencontré quelques difficultés, notamment avec le Scanner (problème de `nextDouble()` suivi de `nextLine()`), Grâce à ce TP, je comprends mieux comment modéliser des objets concrets (client, compte) et comment les faire interagir dans un programme Java.